Final Internship Report

Submitted in partial fulfilment of the internship requirement

Of

Bachelor of Science **Artificial Intelligence**

Student Name	Humna Imran
Registration No.	2023-BS-AI-017
Program	BS Artificial Intelligence
Host Organization	Code Saviours
Internship Duration	29 June 2026 – 29 August 2026
Industry Supervisor	Ms. Rameesha
University Supervisor	Mr. Arham
Submission Date	31 August 2026

Department of Computer Science

The University of Faisalabad

1. Introduction

Code Saviours is the organisation that hosted my internship, under its Artificial Intelligence and Machine Learning internship programme (Batch SI-26). The internship was not a single fixed assignment. It was set up as a structured, project-based programme instead. We were given weekly milestones, and we were expected to keep our work in version-controlled GitHub repositories. We were also guided to take a project from an initial idea all the way to a working, deployed application. Because of this, the internship was closely aligned with how AI and Machine Learning work is actually carried out in practice. It was not just about building models. It also meant collecting data, writing documentation, debugging, and deploying the final product.

During the internship, I worked on two separate projects, both supervised under the same programme. The first was a system to automatically identify Roman Urdu and English words inside code-switched text, which is a Natural Language Processing problem. The second, bigger project was building a custom Optical Character Recognition system for Urdu text, an area that combines computer vision with language processing. Both projects connect directly to my BS Artificial Intelligence degree, since they required me to apply supervised learning, transformer-based deep learning models, honest evaluation, and practical skills such as version control and deployment. The rest of this report describes the work I carried out on both projects, what I learned from them, and how the experience helped me grow as an early-career AI practitioner.

2. Tasks and Responsibilities

Across both projects, my work covered the full lifecycle of an applied Machine Learning project. This meant understanding the problem, finding and preparing data, building and training models, evaluating the results honestly (even when they were disappointing), debugging failures, and packaging the finished work into something another person could use. Each week of the internship had its own assigned deliverable. The two subsections below describe, project by project, what I was asked to do and how I did it. Table 1 lists the main technologies and tools I used for this.

Table 1: Technologies and Tools Used During the Internship

Category	Tools / Technologies Used
Programming language	Python

Deep learning framework	PyTorch, Hugging Face Transformers
Core models	XLM-RoBERTa (token classification), TrOCR / microsoft/trocr-base-printed
Data handling	pandas, NumPy
Computer vision / preprocessing	OpenCV, Pillow
Baseline OCR engine	Tesseract OCR with the Urdu language pack
Visualization	Matplotlib, Seaborn, scikit-learn
Development environment	GitHub Codespaces; Google Colab (GPU training)
Version control	Git and GitHub
Model and dataset hosting	Hugging Face Hub
Deployment	Streamlit / Streamlit Community Cloud

I kept both projects under Git and GitHub the whole time, instead of writing everything up after the fact, so the commit history is itself a record of the work. The Urdu OCR repository has 221 commits made between 30 June and 20 August 2026, across all five of its weekly stages. The code-switching repository has 30 commits made between 7 and 21 August 2026, across its two weekly stages. A good number of these commits are just plain bug fixes, for example correcting notebook execution counts, handling a FileNotFoundError during dataset loading, and repairing a broken package-installation command. This matches the trial-and-error process described throughout the rest of this section. It was not one single polished attempt.

Project 1: Roman Urdu-English Code-Switching Language Identification

The first project was done in Weeks 6 and 7 of the internship. It looked at code-switching: the way many Pakistani social media users mix Roman Urdu and English in the same sentence, and sometimes in the same phrase. Most standard NLP tools assume a document is written in a single language, so this kind of mixed text is hard for them to process correctly. My task was to build a model that reads a sentence and labels each individual word as Urdu (URD), English (ENG), or a mixed/hybrid token (MIX). This makes it a word-level token classification problem, not a sentence-level one.

This kind of model is not just a labelling exercise. It also has fairly direct real-world uses, such as sentiment analysis on mixed-language text, chatbots and keyboard suggestions that do not

break on Roman Urdu, and content moderation systems that currently tend to assume only one language.

Dataset. Instead of starting from scratch, I built the training data from a public Roman Urdu dataset (the Smat26/Roman-Urdu-Dataset collection of real Twitter, Facebook, and e-commerce review text, released under a GPL-3.0 license). I cleaned and de-duplicated the raw text down to about 15,900 usable sentences. Then I auto-labelled every word using an English wordlist, cross-checked against a curated list of common Urdu words that would otherwise get misclassified. After that, I filtered the sentences to keep only the ones that showed genuine code-switching (enough of a mix of Urdu and English words, with English as the minority). This left 220 sentences. I spot-checked these by hand before finalising a dataset of 2,490 word-level labels. Figure 1 shows the label balance.

Building the labelling rule took a few tries to get right. A plain English wordlist is not enough to separate the two languages, because a lot of short, common Urdu words are also valid English words, or are easy to confuse with them. For example, is, mil, beta, and par all caused early mislabelling. So I built a curated override list of more than 150 common Urdu words, to catch these before the English wordlist could claim them. This filter also needed tuning. My first pass returned around 300 candidate sentences, against a target of 220. When I de-duplicated them, I found a subtler bug: the same sentence had been kept twice, because one copy used different capitalisation from the other. Comparing sentences case-insensitively before de-duplicating fixed this. When I spot-checked a random sample of the final labels by hand, the automatic labelling held up well. By my own estimate, it was about 95% correct. The remaining errors were truly ambiguous, not careless mistakes. For example, the word "mobile" sits in the English wordlist, but in Pakistani Roman Urdu it gets used constantly as an ordinary Urdu-feeling word ("mera mobile"). That is exactly the kind of judgement call a fully automatic pipeline cannot make on its own.

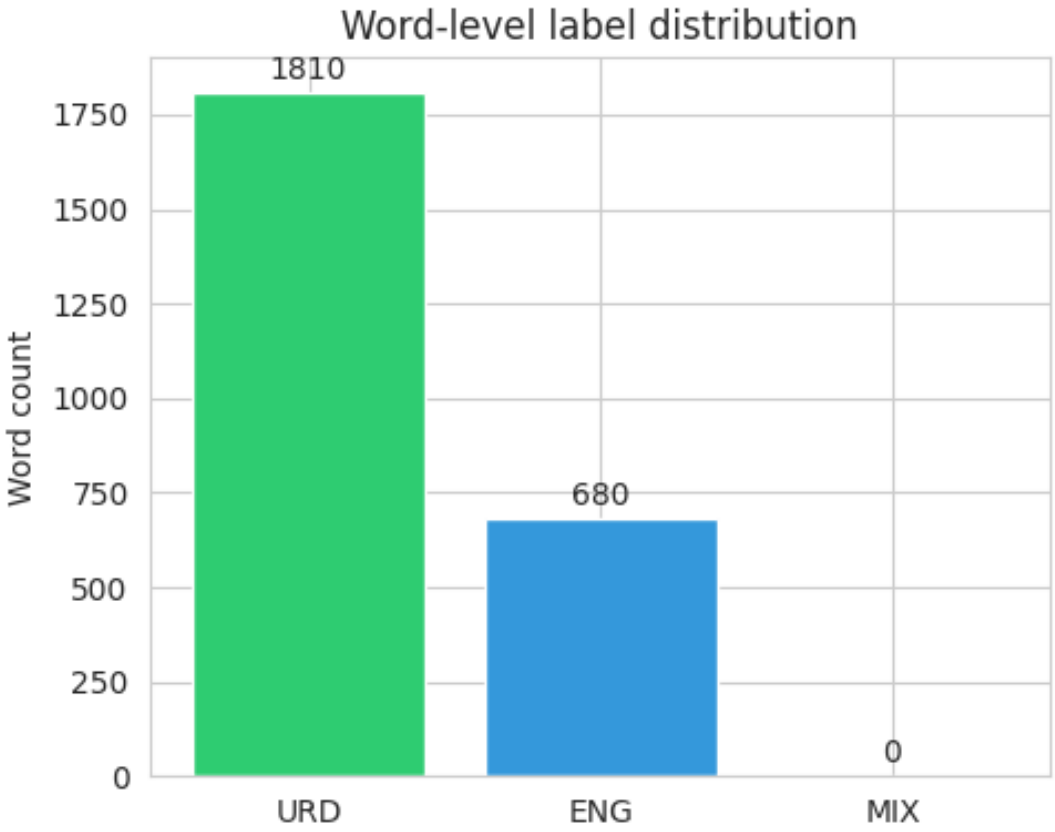

Figure 1: Word-Level Label Distribution in the Code-Switching Dataset

Looking at which words showed up most often in each label also helped me sanity-check the labelling. Figure 2 shows that the most common Urdu-tagged words were everyday grammatical words (hai, ke, ko, mein). The most common English-tagged words were mostly proper nouns and loanwords. This is what I would expect from real, informal, mixed text.

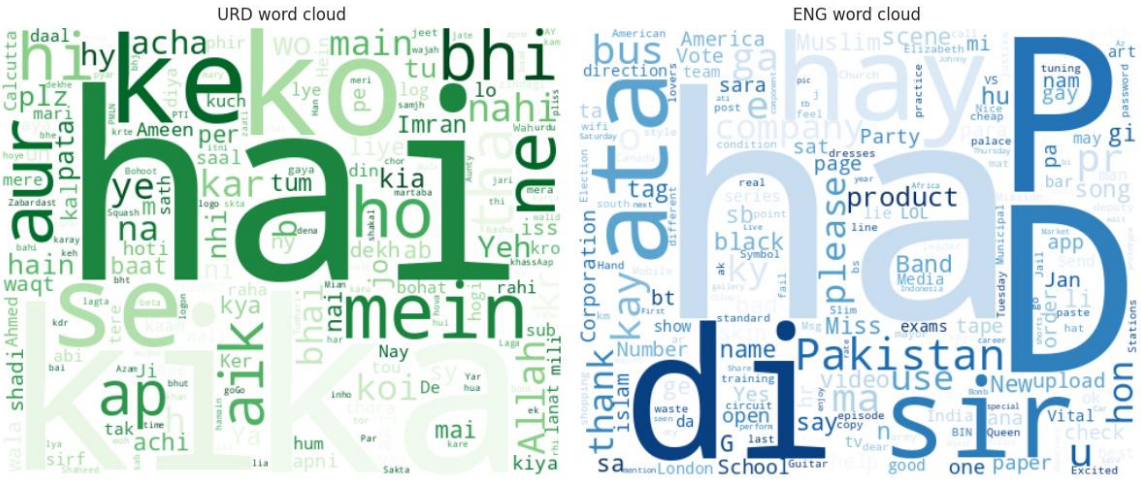

Figure 2: Word Clouds of the Most Frequent Urdu (URD) and English (ENG) Tokens

Model development and training. I fine-tuned xlm-roberta-base, a multilingual transformer model, for token classification across the three labels. Training used an 80/20 split (176 training sentences, 44 held out for testing), 8 epochs, a batch size of 16, and a learning rate of 2e-5. The Hugging Face Trainer automatically picked the best checkpoint based on macro F1 score on the evaluation set. Figure 3 shows that training loss dropped steadily, while macro F1 rose quickly in the first three epochs before levelling off. This suggested the model had converged, given the amount of training data I had. Training ran on Google Colab. On my first attempt the GPU did not attach automatically, so I had to explicitly switch the runtime type to a T4 GPU before training would use it.

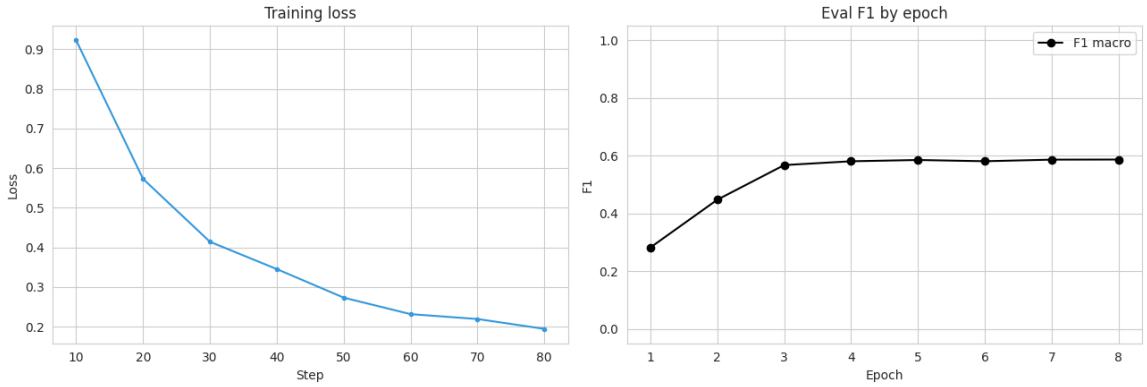

Figure 3: Training Loss and Macro F1 Across Epochs for the Code-Switching Model

Evaluation. On the held-out test set, the model reached 90.75% word-level accuracy. Table 2 breaks this down by label. Performance on Urdu and English words was strong, but the model never predicted the MIX label at all.

Table 2: Precision, Recall, and F1-Score by Label for the Code-Switching Model

Label	Precision	Recall	F1-score
URD	0.933	0.942	0.937
ENG	0.836	0.812	0.824
MIX	0.000	0.000	0.000

Figure 4 shows the same result as a confusion matrix. Almost all of the model's mistakes were mixing up Urdu and English words with each other, for example borrowed or ambiguous words. The MIX row and column are empty.

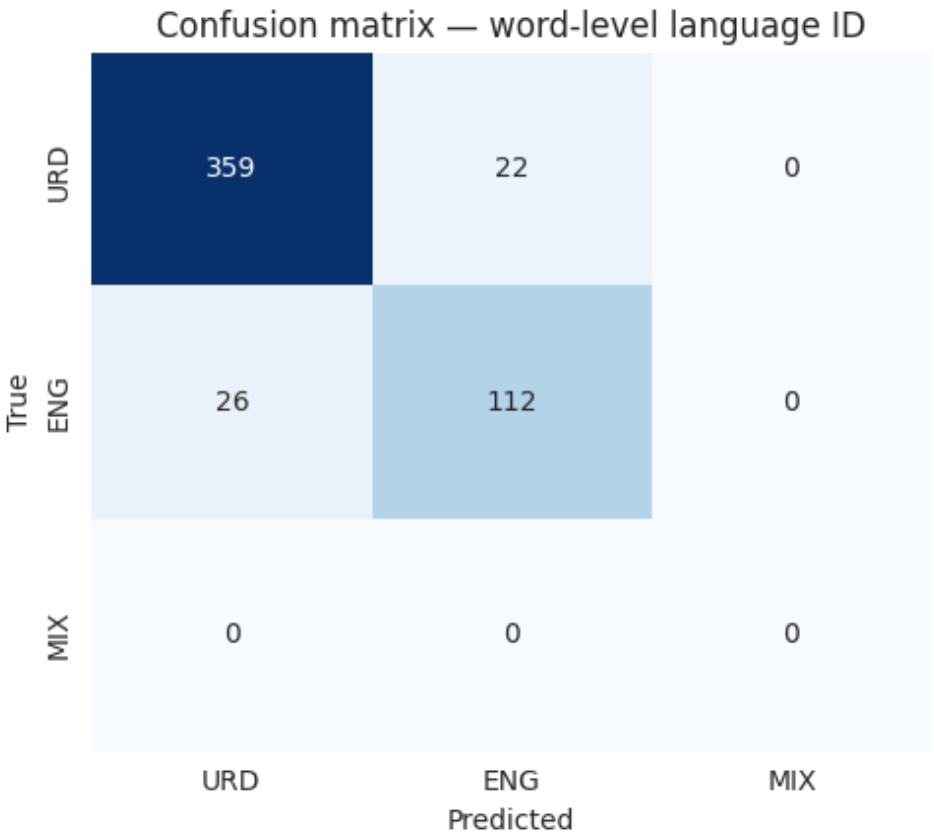

Figure 4: Confusion Matrix for Word-Level Language Identification

Debugging. The fact that MIX never shows up was not really a training failure. It was more of a data problem, and tracing it back was one of the more useful debugging exercises of the internship. My word-level auto-labelling step only accepted tokens that matched a strictly alphabetic pattern. This meant hyphenated hybrid words, which is the most natural way hybrid Roman Urdu-English words show up in real text, got filtered out of the dataset before they ever had a chance to be labelled MIX. In other words, the model correctly learned that MIX almost never occurs, because in its training data it never did. I see this as a real limitation in how I built the dataset, and I am including it here instead of leaving it out. An evaluation section that only shows the flattering numbers would not give a fair picture of what the model can do.

Application and deployment. After training, I published the fine-tuned model and the dataset to the Hugging Face Hub. I also built a small Streamlit application that loads the model directly from the Hub, and lets a user type in a sentence and see each word tagged and colour-coded by predicted language in real time. The documentation went through its own changes alongside the code. The first README, drafted on 7 August with help from GitHub Copilot, was little more than a placeholder. I did not really develop it further until the notebook, the dataset, and

the trained model were finished on 13 August. Once the model and app were working, I rewrote the README properly. This meant fixing a Colab badge link that had pointed at the wrong notebook path, replacing a placeholder Hugging Face dataset link with the real one, and reorganising the whole document around the finished product (live demo, results, how to run it) instead of around the research process behind it. Afterward, I also went back through the repository and removed an old assets folder and some old notebook versions left over from earlier drafts. A repository that is easy for someone else to navigate felt like part of finishing the project properly, not an optional extra.

Figure 12 shows the deployed application correctly tagging a live example sentence. Figure 13 shows the model's page on the Hugging Face Hub.

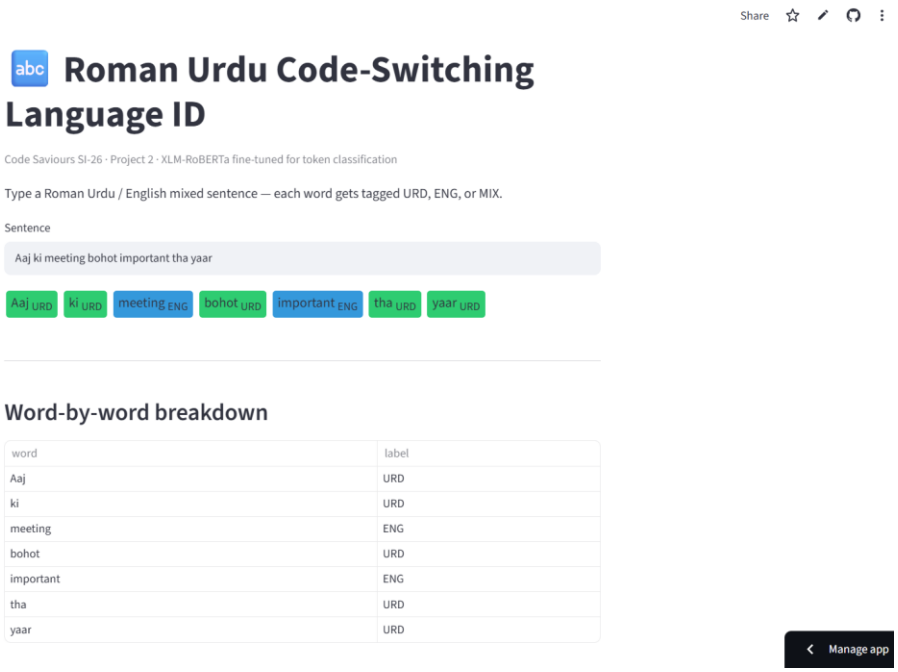

Figure 12: Screenshot of the Deployed Code-Switching Streamlit Application

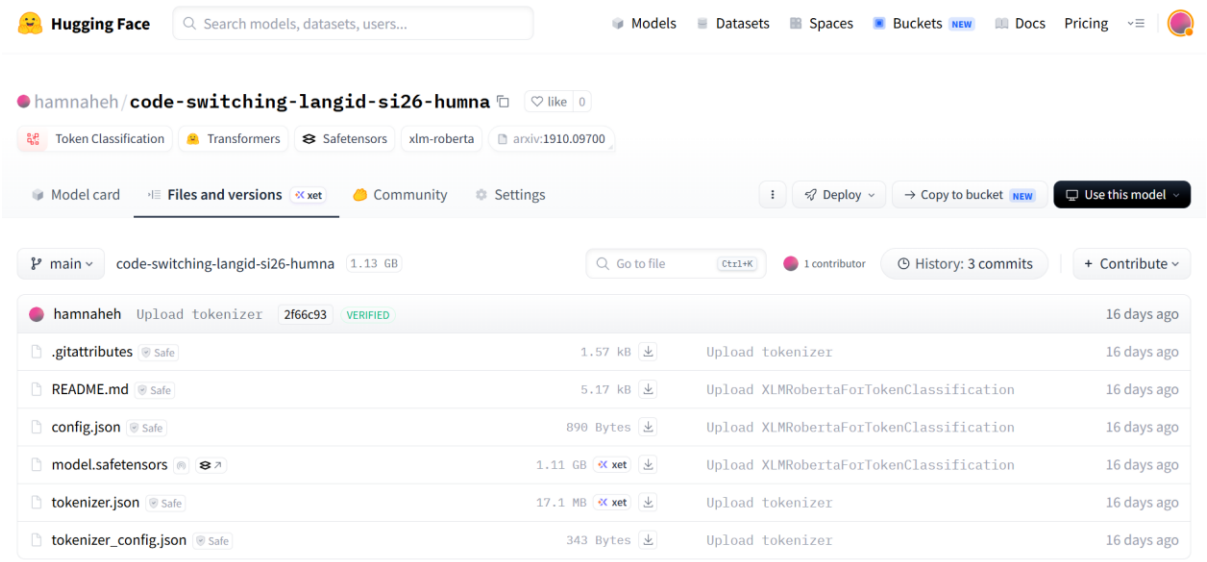

Figure 13: The Code-Switching Model's Page on the Hugging Face Hub

Project 2: Urdu Optical Character Recognition (OCR)

The second project ran across Weeks 1 to 5 of the internship, and it was a lot bigger in scope: building a custom OCR system for Urdu text. Urdu is normally written in the Nastaliq script. This script is highly cursive, flows diagonally instead of sitting on a straight horizontal line, and changes each letter's shape depending on where it sits in a word. On top of that, there is far less labelled Urdu OCR data publicly available compared to English. This makes Urdu OCR a substantially harder problem than English OCR, and it is also a good example of why off-the-shelf tools do not always work well across different languages and scripts.

Data collection. Week 1 was mostly about building a labels.csv schema (image, text, category) that every later week would build on, and then filling it in. I used UTRSet-Real, an academic dataset of real scanned Urdu text lines, and took 60 of its lines as real-world ground truth. I also rendered an initial batch of 20 synthetic Urdu line images by hand, just to give the dataset something to start with before I had a proper rendering pipeline. For the other real-world categories, I scanned book pages from the Urdu novel Jannat Ke Pattay, and took newspaper images at first from Akhbar-e-Jahan, Jang, and Dawn News. I noticed these newspaper images were inconsistent in aspect ratio and quality compared to the rest of the dataset, so I replaced that set with pages from Hamdard Naunehal instead. Typing out the Urdu transcription for every image by hand turned out to be far slower than I expected, since several of the book and

newspaper captions ran to full paragraphs. So instead, I uploaded the images to Google's Gemini one at a time, used it to transcribe the text, checked each result against the image myself, and pasted the confirmed transcription into labels.csv. I also ran into a more mundane problem here: some transcriptions had commas and quotation marks that broke the CSV format, and I had to go back and fix these by hand.

Image preprocessing. My first preprocessing pipeline used a fixed brightness threshold, and forced every image into the same fixed size. It did not work well. On real photographed pages with uneven lighting, the fixed threshold turned shadowed regions into speckled noise. Forcing a uniform size also distorted the aspect ratio of images that were naturally very different shapes, like single cropped text lines versus full book or newspaper pages. So I rebuilt the pipeline to resize each image while keeping its aspect ratio, denoise it gently, and then binarize it using Otsu's method. Otsu's method recalculates the ideal brightness cutoff for each image on its own, instead of assuming every photo has the same lighting. Figure 5 compares raw and processed images side by side. Figure 6 shows why Otsu matters in practice: for one processed page, Otsu automatically picked a threshold of 177, far from the old fixed cutoff of 127. That difference would have badly misclassified that image's pixels under the original approach.

Two smaller issues showed up once this pipeline was running. First, my notebook's working directory was not always the folder I expected, so raw images were not loading from the right place. I added a small diagnostic cell that searches the workspace for folders named "raw" and reports how many images it finds, then pointed RAW_DIR at the correct path. This sounds like a trivial fix, but it saved me a lot of confused debugging. Second, Otsu is not foolproof. On a few images with a lot of white padding or border around the actual text, it picked a threshold that turned almost the entire image black. I caught this by adding an automated check that measures the percentage of black pixels in every processed image, and flags anything at the extremes, instead of scrolling through all 153 images by eye. Separately, I also had to clean up the repository itself partway through. A batch of processed PNG files had been committed without filtering, and had made the repository noticeably larger than it needed to be. I removed the unnecessary binaries, kept only the files needed to reproduce the pipeline, and added basic error handling so invalid or corrupted images get moved aside instead of silently breaking a later cell.

Figure 5: Sample Images Before and After the Preprocessing Pipeline

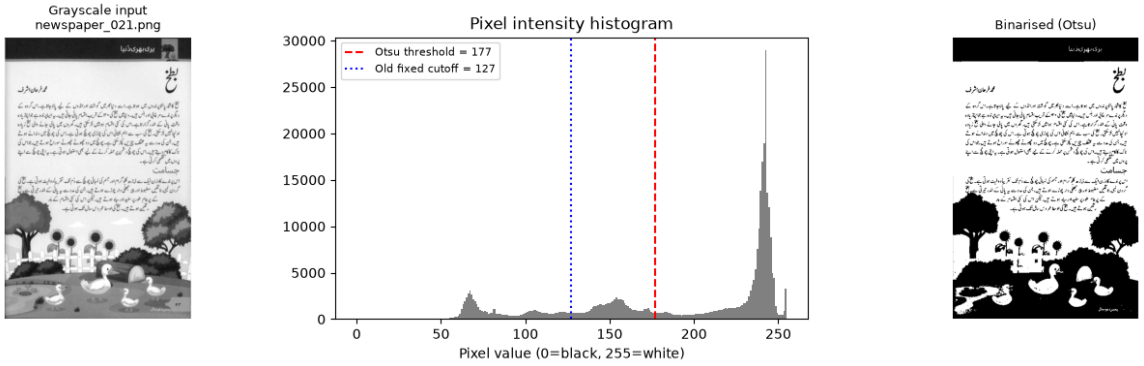

Figure 6: Otsu's Adaptive Threshold Compared with the Original Fixed Cutoff

Baseline evaluation with Tesseract. Before building a custom model, I tested Tesseract, a widely used open-source OCR engine, on five of my processed images using its Urdu language pack. I wanted to understand how much a custom model was really needed. The results were poor. On the two images where I had exact ground truth to compare against, Tesseract recovered only 27% and 0% of the real words (Figure 7). Looking across all five images, one short, clean line survived mostly intact, one produced only a single recoverable fragment, and the two full-page images were the worst. Tesseract regularly output characters and digits that were not anywhere in the source image, which is a different and more worrying kind of failure than just missing a few words. This baseline gave me clear, evidence-based proof of why the project needed a custom-trained model instead of an existing OCR tool.

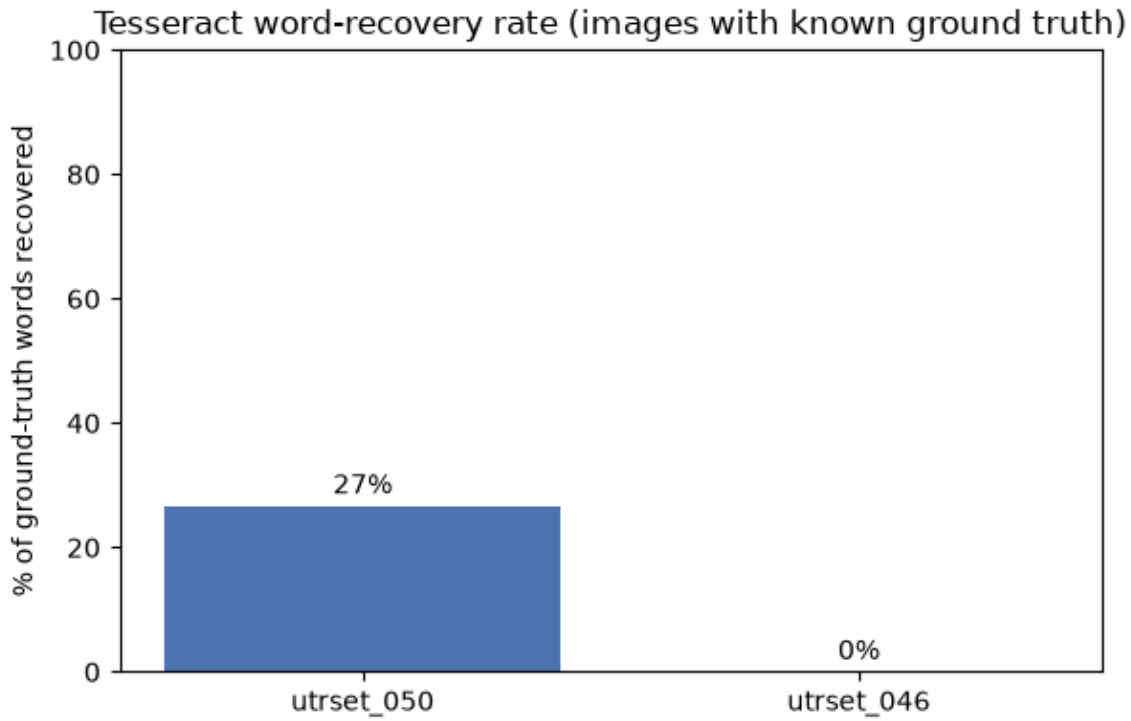

Figure 7: Word-Recovery Rate of Baseline Tesseract OCR on Urdu Text

Dataset expansion. To get more training data, I looked into UTRNet's UTRSet-Real and UTRSet-Synth datasets (academic datasets of real and synthetic Urdu text lines, released for research use). I built a proper rendering engine using the Nastaliq and Naskh fonts, over a public corpus of roughly 137,000 Urdu headlines, with right-to-left aware text shaping so letters joined correctly instead of sitting apart. This gave me 110 additional synthetic Urdu text-line images, and brought the working dataset up to 263 labelled image-text pairs. Wiring this into a proper PyTorch Dataset class turned into its own small project. Some images failed to load because a handful of files were corrupted, so I handled this with try-except blocks. The tokenizer needed padding set explicitly to max_length, or batches came out with mismatched shapes. The pixel tensors also needed an extra squeeze() to come out in the [3, 384, 384] shape the model expected. I also had to pin the transformers library to version 4.57.6, since the newer 5.x release had a backwards-compatibility issue with TrOCRProcessor that broke this notebook.

Model development and training. I fine-tuned microsoft/trocr-base-printed, a transformer-based encoder-decoder OCR model with roughly 334 million parameters, on the 263-image dataset (211 for training, 52 for testing, an 80/20 split). Figure 8 shows this split, along with the spread of ground-truth caption lengths, which ranged from short headlines to full transcribed pages of several thousand characters. Training ran for 3 epochs with a batch size of

4 and a learning rate of 5e-5. Figure 9 shows that training loss dropped a lot over the first few batches, then levelled off at around 3.5.

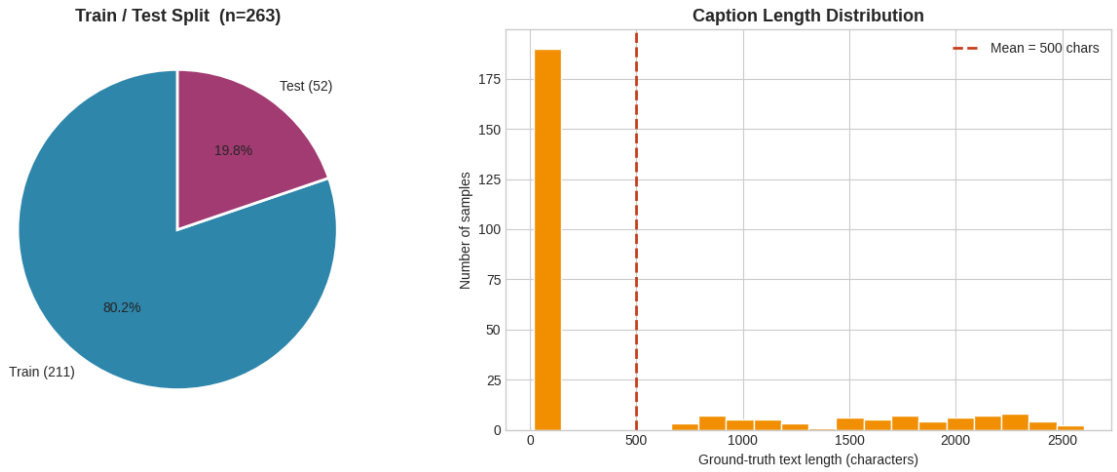

Figure 8: Train/Test Split and Ground-Truth Caption Length Distribution

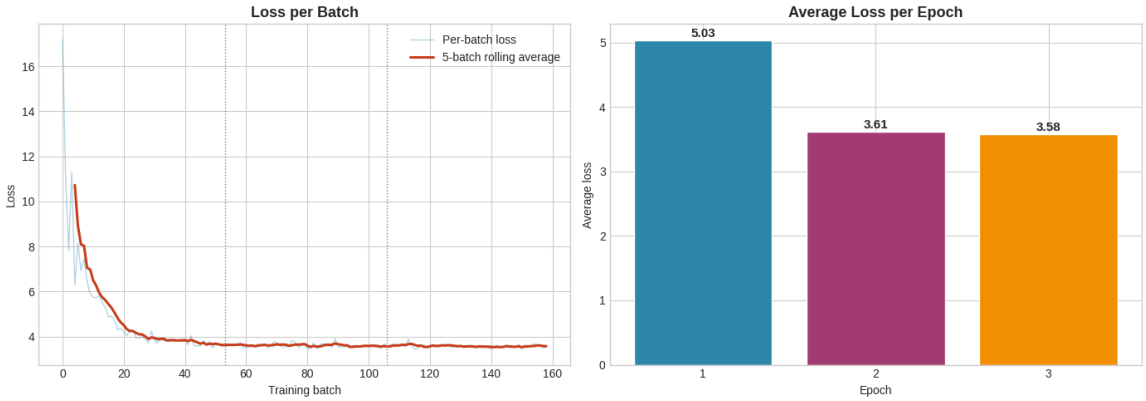

Figure 9: TrOCR Fine-Tuning Loss Across Training

The Week 4 data bug. Before I could even judge the model, I ran into the single most instructive bug of this whole project. I compared the number of rows in labels.csv against the number of image files that actually loaded, and found that 153 of 263 rows, 58 percent of the dataset, were being silently skipped during training. There was no error and no warning. The training loop just ran on whatever happened to load. I traced the path-joining logic by hand, and found that labels.csv had picked up two different, incompatible path conventions from different weeks of data collection. My loader only handled one of them. I wrote a resolve_image_path() function that tries the direct path first, and falls back to the older convention if that fails. This brought the dataset back to its full 263 rows. Finding this bug had less to do with any single line of code, and more to do with a habit. I only caught it because I

had started comparing row counts against file counts as a matter of course, instead of assuming a notebook running without errors meant it was running correctly.

Evaluation. On the 52 held-out test images, the fine-tuned model reached 0% exact-match accuracy, and a mean character-level similarity of only about 0.1%. Every single prediction collapsed to the exact same fixed string of placeholder characters, no matter what the input image contained (Figure 10). Figure 11 shows this directly, against sample test images and their ground truth. The output length stayed constant at 20 characters, whether the true caption was 22 characters or 110. This is the signature of a model that has collapsed, not one that is partially working. Working backward from this, I traced the most likely causes to three things. First, trocr-base-printed's decoder is a RoBERTa model pretrained almost entirely on English text, so it starts out with very little representation of Urdu script. Second, there is a domain gap between the mix of scanned, photographed, and synthetic images in my dataset, and whatever printed text the base model originally saw. Third, 211 training examples across only 3 epochs is a small amount of signal to teach a model an entirely new writing system from close to zero. Further documentation on the project also points to a repeated-character degeneration pattern, consistent with exposure bias, which is a known failure mode in sequence generation models. It also describes later training runs with more epochs and generation-time adjustments, like a repetition penalty, that reportedly brought the validation character error rate down toward 1.365. At the time of writing, that follow-up work is a documented next step, not a result I can re-verify in this notebook. So I have reported the confirmed figures above, instead of a number I could not check myself.

Besides the dataset bug, a handful of environment issues also shaped how I worked during this part of the project. The corpus file that fed the synthetic renderer was sometimes not found at all, because my notebook used a path that did not resolve the same way on every machine. I added an existence check with a fallback and a clear error message, instead of letting it fail somewhere else with a confusing traceback. Rendering Urdu text with right-to-left shaping depended on a system library called libraqm, which is not always installed in a fresh Colab runtime. So I added a check, PIL.features.check("raqm"), that uses proper RTL shaping when the library is available, and falls back to plain text layout when it is not, instead of crashing outright. Colab recycles its runtime and clears local files every few hours, so I mounted Google Drive and pointed the repository clone and font files at a persistent Drive folder, so checkpoints and downloaded assets survived between sessions instead of disappearing. I also found that package installs were sometimes going to a different Python interpreter than the one the notebook was running on. Switching to an explicit sys.executable -m pip install pattern fixed

the import errors that had been coming from that mismatch. None of these fixes were glamorous, but each one had already cost me a confused debugging session before I figured out what was happening.

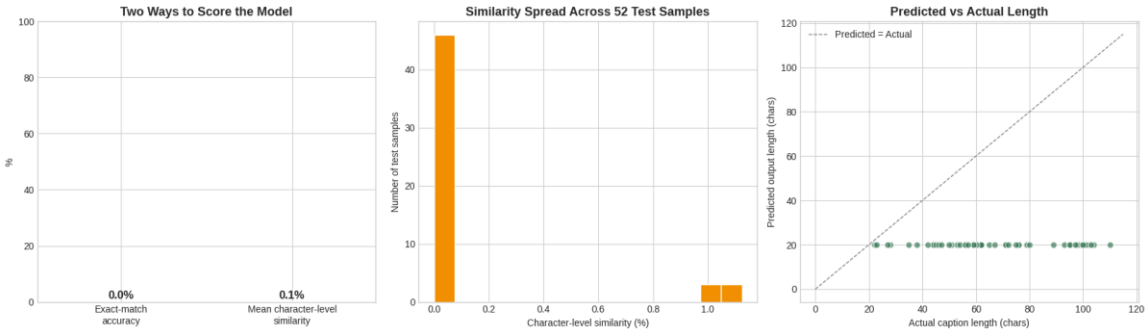

Figure 10: Evaluation of the Fine-Tuned OCR Model on the Held-Out Test Set

Figure 11: Sample Test Predictions Compared with Ground Truth

Application and deployment. Despite the model's limitations, I still built and deployed a complete application around it. It is a Streamlit app that accepts an uploaded Urdu image, preprocesses it, runs it through the fine-tuned model pulled live from the Hugging Face Hub, and shows the recognised text. Getting there took its own round of fixes. The app's MODEL_ID was left as a placeholder for most of Week 5, while training was still running, and the README's results table just said "fill in". Once the final model was finally pushed to the Hub under hamnaheh/trocr-urdu-si26-week4, updating that one line felt like a small change. But it

was the real milestone. It meant training had finished, and a working model now existed for the app to load, not just a plan for one. Being open about an imperfect model, instead of only deploying projects that work perfectly, felt like a realistic and valuable part of the experience. Production Machine Learning systems rarely work well from the start anyway.

Figure 14 shows the application's upload screen. Figure 15 shows the model's actual output on a real uploaded test image, reproducing the same repeated-character collapse described above, outside of the notebook environment. Figure 16 shows the project's GitHub repository, which matches the commit history described earlier in this section.

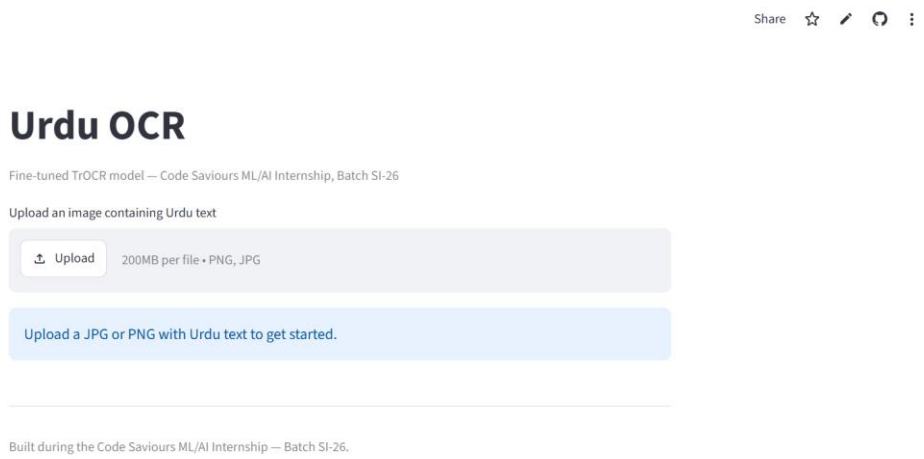

Figure 14: Screenshot of the Deployed Urdu OCR Streamlit Application

Figure 15: The Deployed Model Producing the Same Repeated-Character Output on a Live Test Image

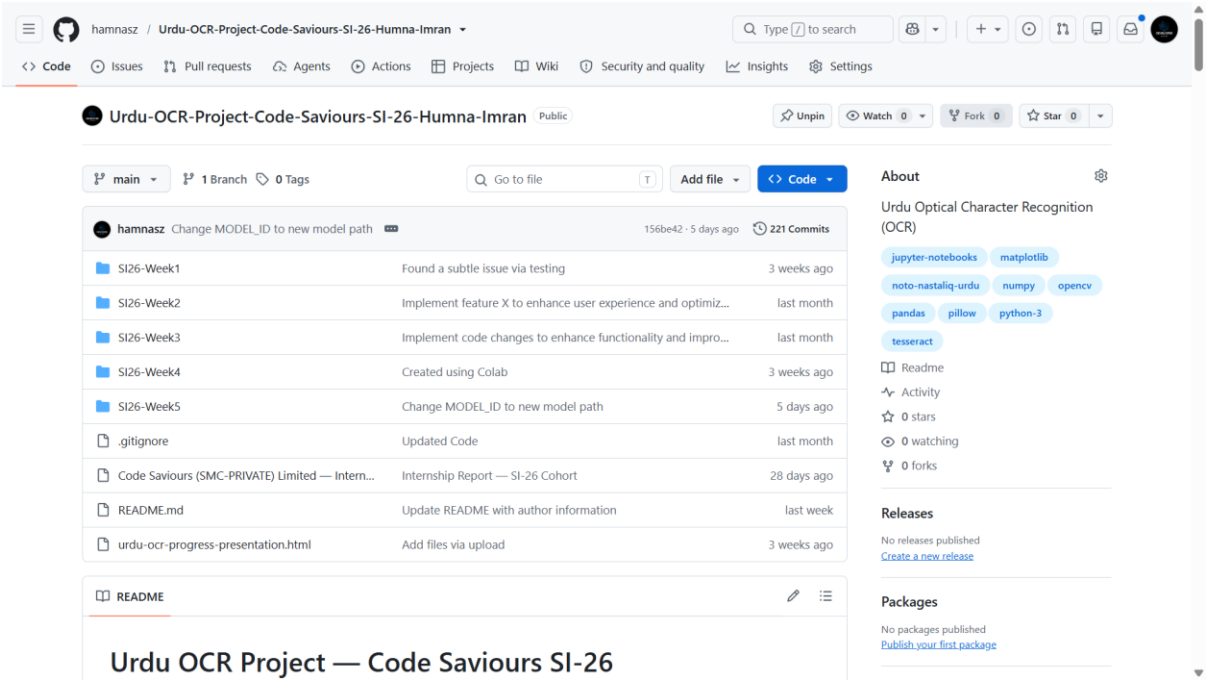

Figure 16: The Urdu OCR GitHub Repository Showing the Weekly Folder Structure and Commit History

Table 3: Comparison of the Two Internship Projects

Aspect	Project 1: Code-Switching ID	Project 2: Urdu OCR
Task type	Word-level token classification (NLP)	Image-to-text generation (vision + NLP)
Base model	xlm-roberta-base	microsoft/trocr-base-printed
Dataset size	220 sentences / 2,490 word labels	263 image-text pairs
Final result	90.75% word-level accuracy	0% exact-match (failure diagnosed and documented)
Deployment	Streamlit app + Hugging Face Hub	Streamlit app + Hugging Face Hub

3. Learning Outcomes

Working on these two projects gave me practical experience with almost the entire Machine Learning workflow, more than lecture-based coursework in my BS Artificial Intelligence degree could offer on its own. There is a real difference between reading about data cleaning, tokenisation, or train/test splitting in a course and writing the code that filters 15,000 sentences down to a useful 220, and an even bigger difference between reading about it and discovering that a filtering rule I had written myself had quietly deleted an entire label from my own dataset.

On the Natural Language Processing side, I learned how to approach code-switching as a concrete labelling problem, how to fine-tune a multilingual transformer model (xlm-roberta-base) for token classification using the Hugging Face Trainer API, and how to read training curves and confusion matrices critically instead of just checking whether the overall accuracy number looked acceptable.

On the computer vision and OCR side, I learned that classical image preprocessing choices, such as thresholding method and resize strategy, are not minor implementation details: they can be the difference between usable and unusable input data. Fine-tuning a pretrained model on a script it has barely seen turned out to require a lot more than running a training loop for a few epochs, and small dataset sizes matter enormously when a model has to learn new visual and linguistic patterns from close to zero.

Beyond the two specific projects, I also became noticeably more comfortable with the tooling around real Machine Learning work: structuring a project across GitHub Codespaces and Google Colab, using Git and GitHub properly for version control across many weekly notebooks, publishing models and datasets to the Hugging Face Hub, and turning a notebook-based result into a deployed Streamlit application someone else could open and use. These are skills that are easy to underweight in an academic setting, but they were a large part of what made this feel like a real project and not a classroom exercise.

4. Professional Growth and Reflections

The internship pushed me to work far more independently than most of my university coursework does. There was no instructor checking each step in real time, so when something broke, like the OCR model collapsing to a single repeated output, diagnosing it and deciding what to do next was on me. That was genuinely hard at times. When a result was bad, I often could not immediately tell whether the cause was a bug in my code, a limitation of the dataset, or a limitation of the model itself, so I had to build a habit of checking each possibility systematically instead of guessing.

I also got better at managing a project across multiple weeks instead of in a single sitting. Both projects were structured as a sequence of weekly milestones, and staying organised across notebooks, data folders, and changing file paths, which was the direct cause of one of the OCR bugs, taught me the value of consistent naming conventions and clear documentation. It stopped being an abstract best practice once I felt the cost of getting it wrong firsthand.

The biggest shift for me was in how I handle a disappointing result. My industry supervisor said something that stuck with me: a 0% accuracy score is fine, as long as I can explain why it

happened. A year ago, I would probably have buried the OCR model's 0% exact-match score in a footnote and moved on. This time, I traced it back to a plausible cause and put the real number in the report, where anyone could see it, which felt more useful than showing only the parts that worked. I do not think this internship made me an expert in NLP or OCR, but it gave me a far more realistic sense of what applied AI work involves, including the parts that do not go as planned.

Separately, I learned that training a model and shipping something someone else can click on and use are two different skills. Getting the code-switching project from a working notebook to a live link anyone could open took a different kind of effort than getting the model to 90% accuracy in the first place, and I do not think I appreciated that distinction before this internship.

Between the two projects, I went from auditing a folder of 100-odd images in Week 1 to defending a 0% result with a root-cause explanation in Week 4, and from a fine-tuned model sitting in a notebook to two links anyone can click. That range is probably the best summary of what changed for me over these eight weeks.

5. Recommendations

The weekly milestone structure used for both projects worked well for keeping the work paced and manageable, and I would recommend keeping it for future interns. One addition that might help future cohorts is a short mid-internship checkpoint focused specifically on data pipeline correctness: checking label distributions and file path consistency early. Several of the more time-consuming bugs I ran into, in both the code-switching label filter and the OCR file paths, would likely have surfaced sooner with a dedicated review step, instead of turning up later during model evaluation.

For the OCR project specifically, the problem turned out to be harder than a three-epoch fine-tuning run could address. It may be worth telling future interns upfront that a first attempt at fine-tuning a model on a low-resource script is likely to fail outright, so they can plan for a longer, more iterative training process from the start, instead of treating an initial failed run as a surprise.

More generally, I would recommend that future projects in this programme build in some dedicated time for baseline comparisons, like the Tesseract baseline I ran before building the custom OCR model. That step was one of the most useful things I did: it gave me clear, defensible evidence for why the custom model was worth building, instead of just assuming it was.

6. Conclusion

This internship let me take two Artificial Intelligence projects, one in Natural Language Processing and one in Optical Character Recognition, from an initial idea through data collection, preprocessing, model training, evaluation, and deployment. The Roman Urdu-English code-switching model reached a solid 90.75% word-level accuracy and runs as a working Streamlit application. The Urdu OCR project's fine-tuned model did not produce usable output, but diagnosing why, and documenting it clearly, taught me just as much as a clean result would have.

Across both projects, I applied and deepened concepts from my BS Artificial Intelligence coursework, including transformer-based model fine-tuning, evaluation methodology, and classical image preprocessing, while building practical skills in version control, cloud-based model hosting, and application deployment that a purely academic setting does not always provide. Just as importantly, I came out with a more realistic, professional attitude toward Machine Learning work: an unflattering result is something to investigate, not something to hide. I expect both the technical skills and this outlook to carry directly into my continued studies and any AI-related work I take on after this.

Appendices

Appendix A: Internship Offer Letter

codesaviours

SMC-PRIVATE LIMITED

Issue Date: 25 June 2026

339-C GM Abad, Opp. Govt Teaching Hospital, Faisalabad

codesaviorsteam@gmail.com | @codesaviours_smc_pvt_ltd

INTERNSHIP OFFER LETTER

TO

Humna Imran

Dear Humna Imran,

We are pleased to offer you an internship opportunity at Code Saviours (SMC-PVT) Ltd as a Machine Learning Intern. This internship is designed to provide you with practical experience and exposure to real-world projects within a professional environment.

Internship Details

Internship Title:	Machine Learning Intern
Department:	Artificial Intelligence
Duration:	2 Months
Start Date:	29 June 2026
End Date:	29 August 2026

Expectations

During your internship, you are expected to:

- Participate actively in assigned projects and tasks.
- Maintain a professional attitude and adhere to company policies.
- Submit any required reports or project outcomes by the end of the internship.

Completion

Upon successful completion of the internship, you will be awarded an official Internship Completion Certificate from Code Saviours (SMC-Private) Limited.

We look forward to your contribution and hope this internship will be a valuable step in your career development.

Sincerely,

CODE SAVIOURS (SMC-PRIVATE) LIMITED C.E.O

Rameesha

Chief Executive Officer (CEO)

Code Saviours (SMC-Private) Limited

Code Saviours (SMC-Private) Limited | 339-C GM Abad, Opposite to Govt Teaching Hospital, Faisalabad

codesaviorsteam@gmail.com | @codesaviours_smc_pvt_ltd

Figure 17: Internship Offer Letter from Code Saviours (SMC-Private) Limited

Appendix B: Internship Completion Certificate

Figure 18: Internship Completion Certificate from Code Saviours (SMC-Private) Limited

Appendix C: Supporting Documents

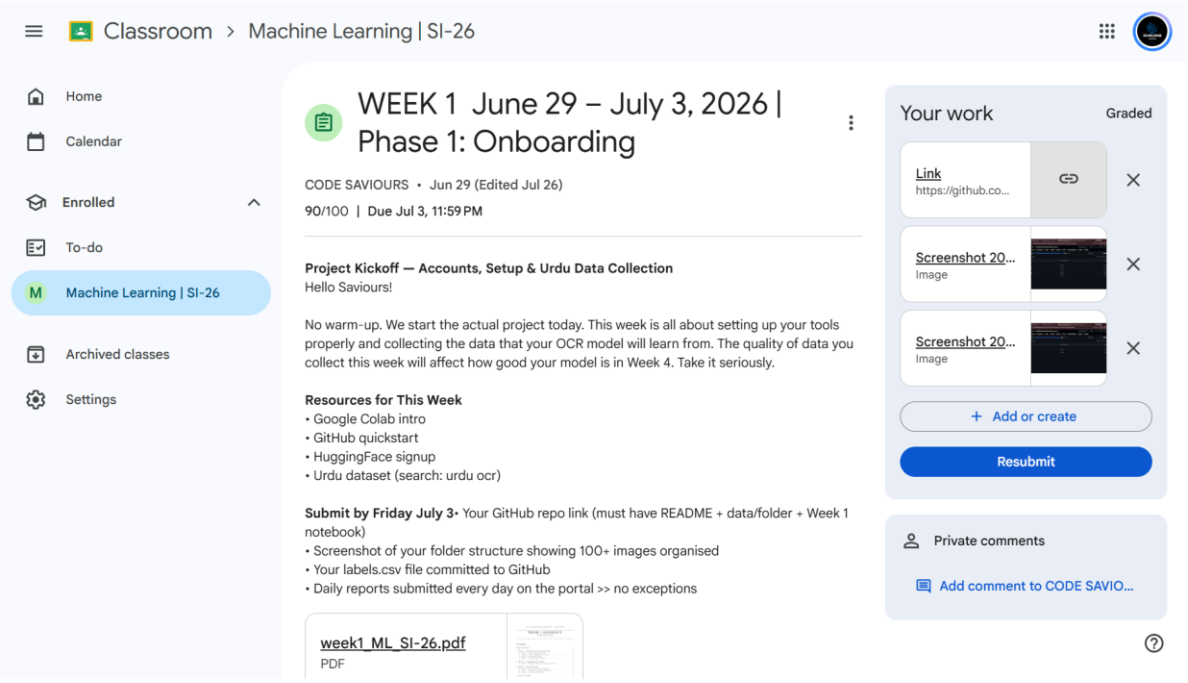

Figure 19: Google Classroom Record for Week 1, Urdu OCR Project: Data Collection (Graded 90/100)

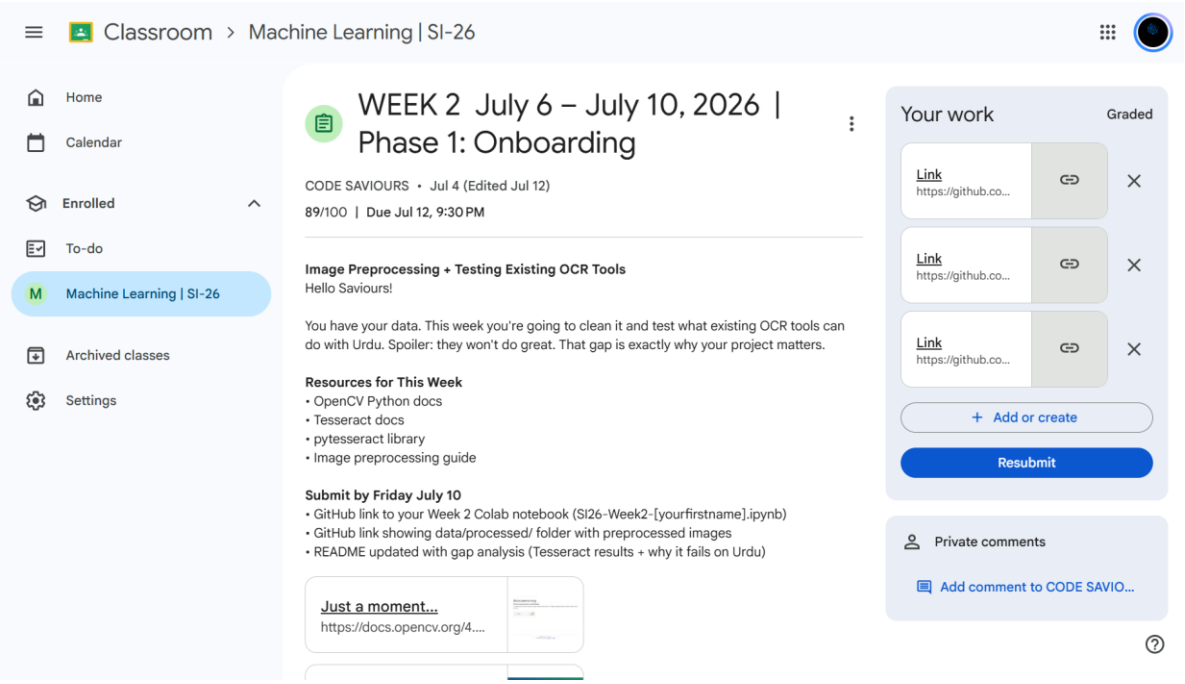

Figure 20: Google Classroom Record for Week 2, Urdu OCR Project: Preprocessing and Baseline Testing (Graded 89/100)

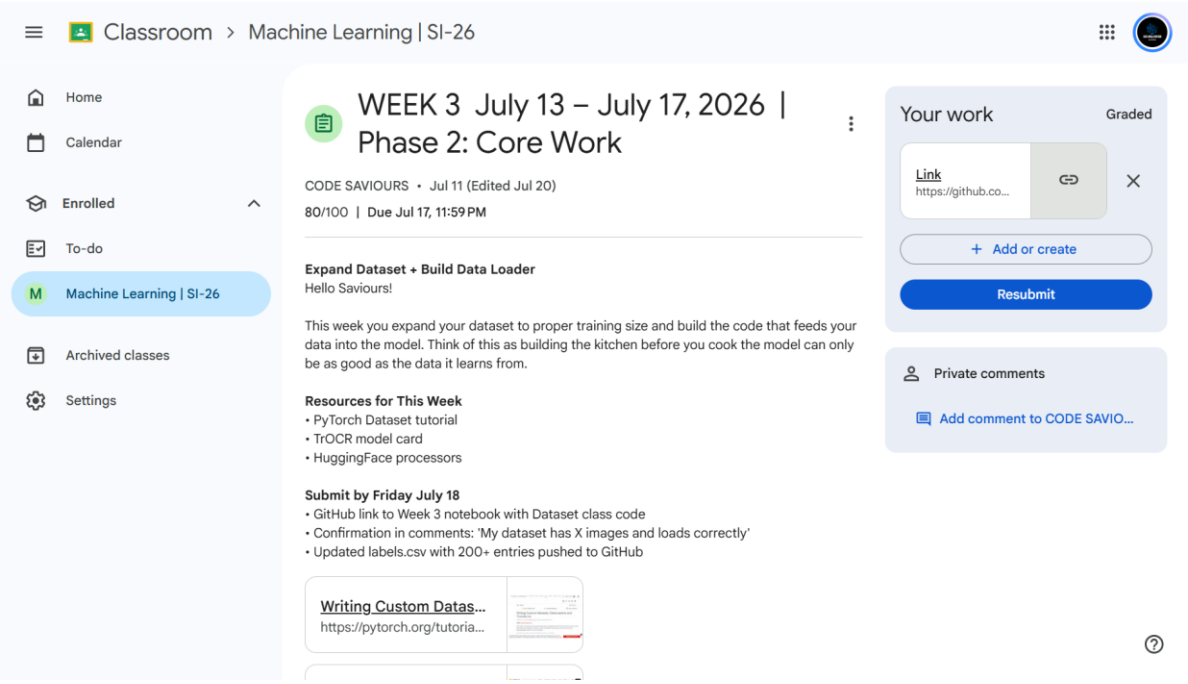

Figure 21: Google Classroom Record for Week 3, Urdu OCR Project: Dataset Expansion and Data Loader (Graded 80/100)

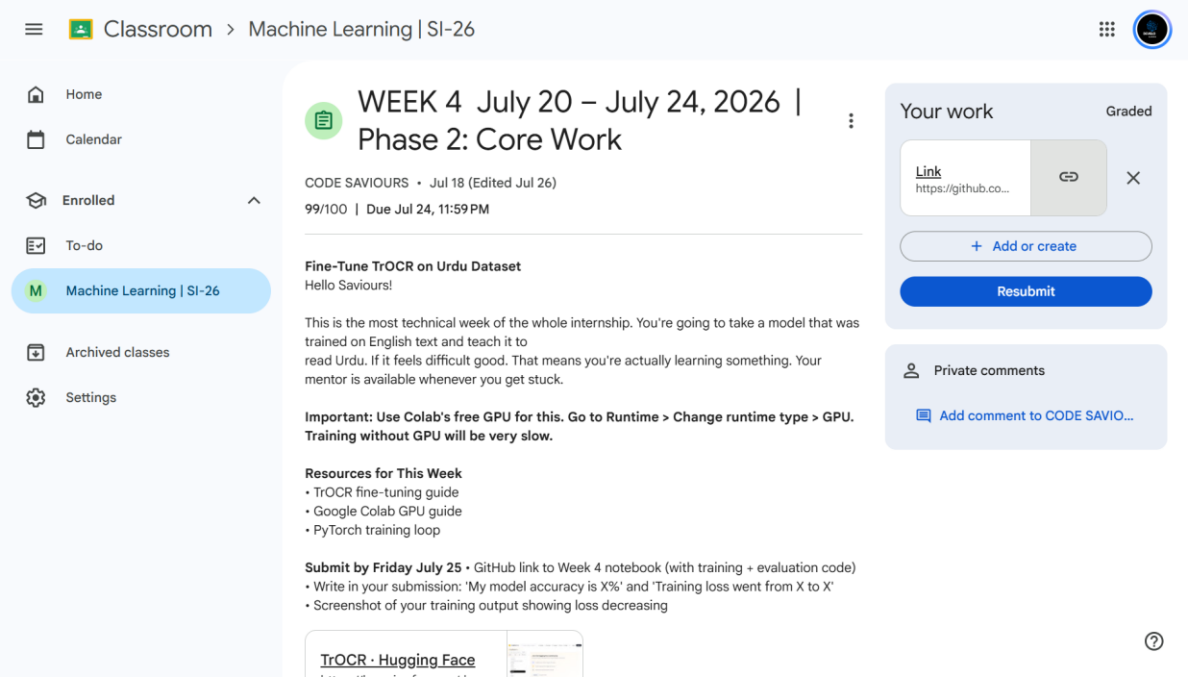

Figure 22: Google Classroom Record for Week 4, Urdu OCR Project: Model Fine-Tuning (Graded 99/100)

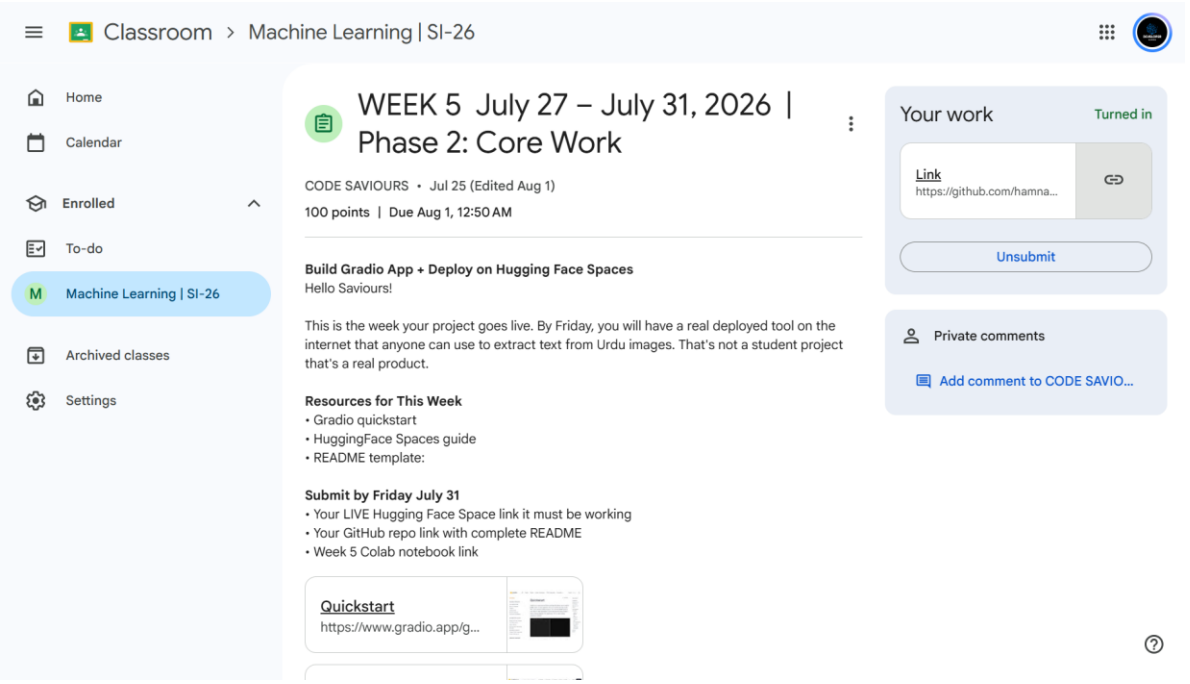

Figure 23: Google Classroom Record for Week 5, Urdu OCR Project: Debugging and Deployment

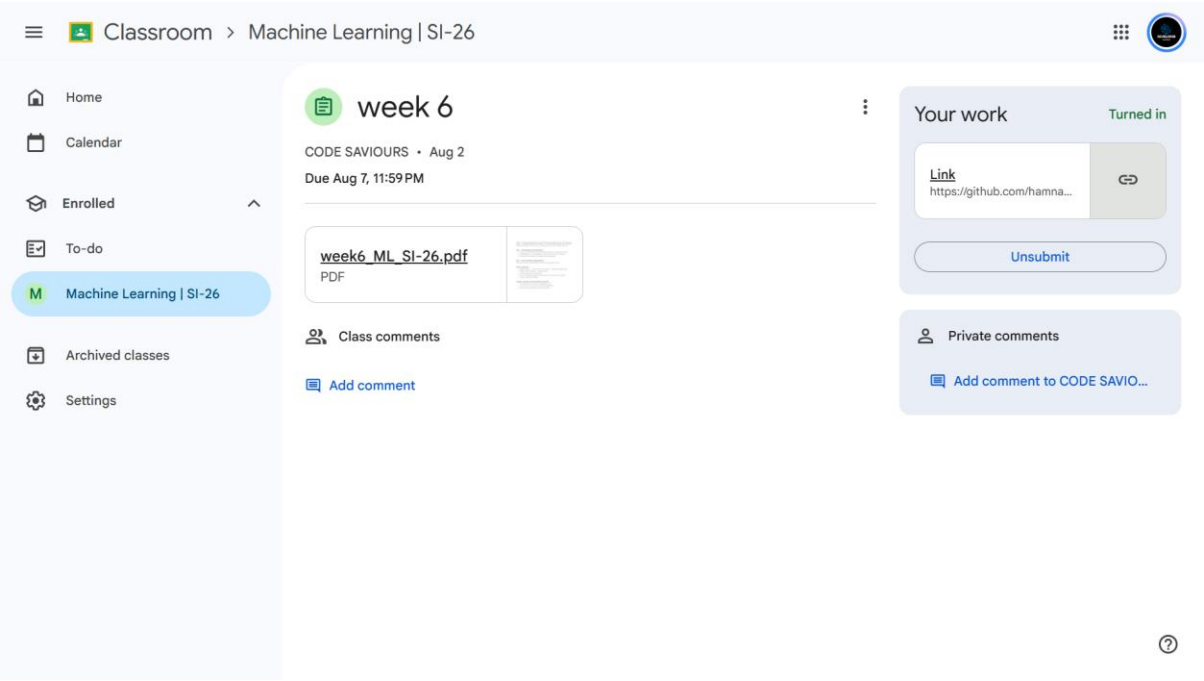

Figure 24: Google Classroom Record for Week 6, Code-Switching Project: Dataset and Documentation

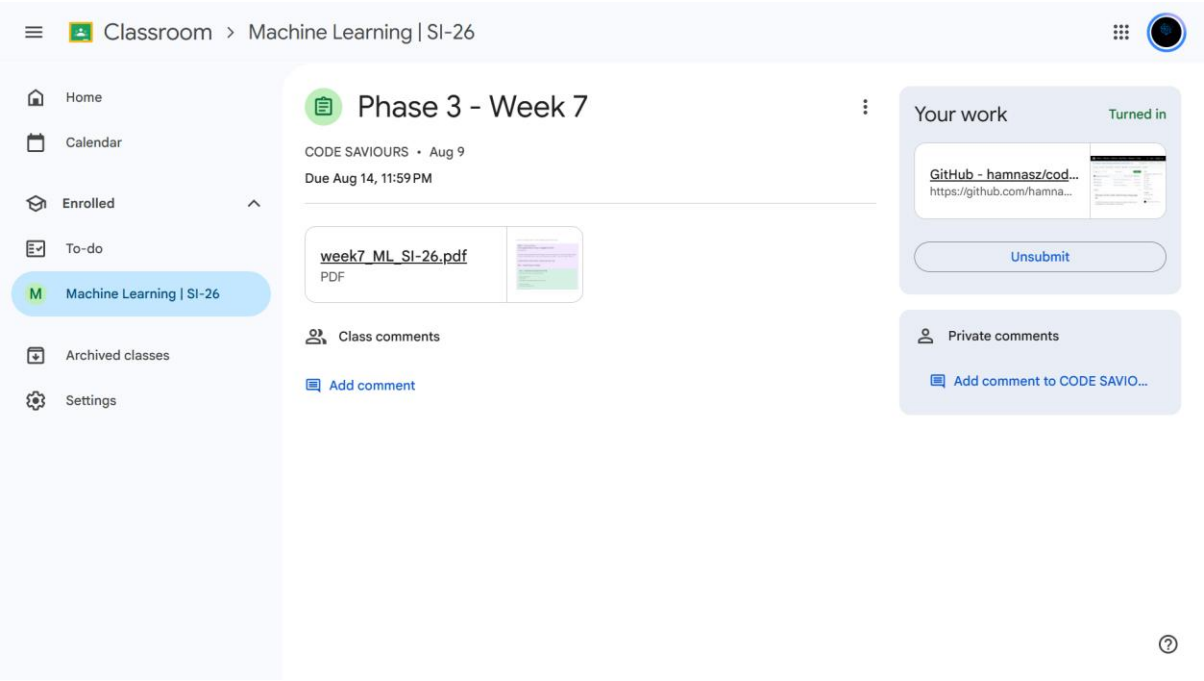

Figure 25: Google Classroom Record for Week 7, Code-Switching Project: Model Training and Deployment

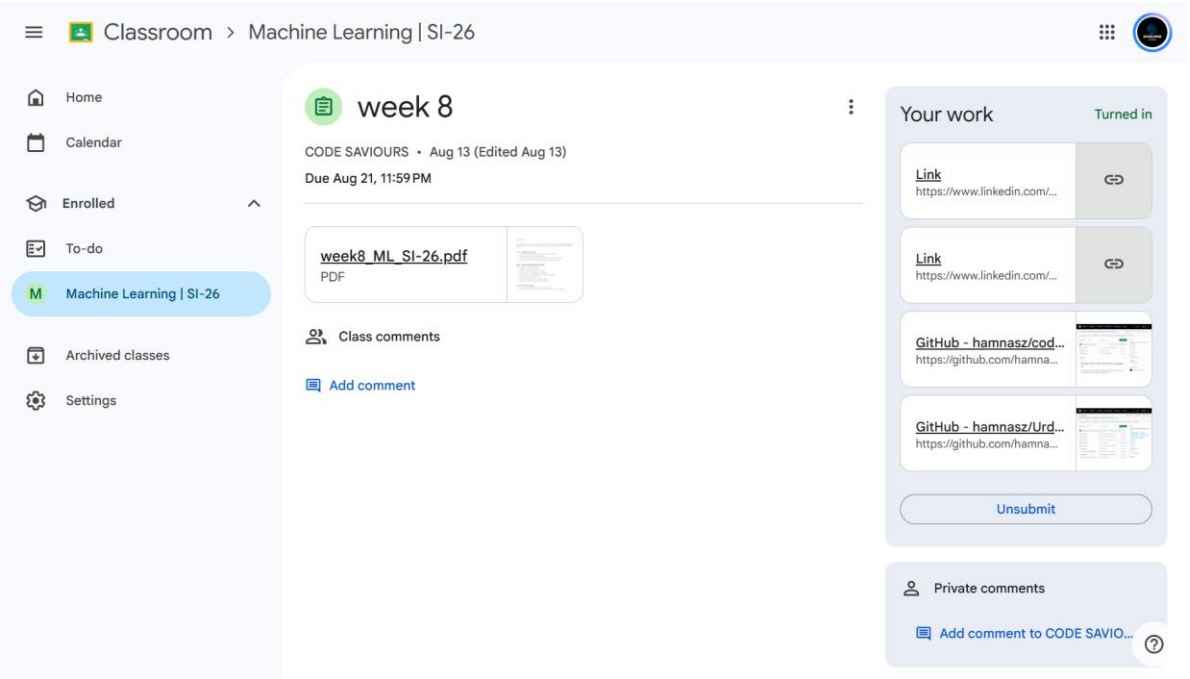

Figure 26: Google Classroom Record for Week 8: Final Submission, Including GitHub and LinkedIn Links

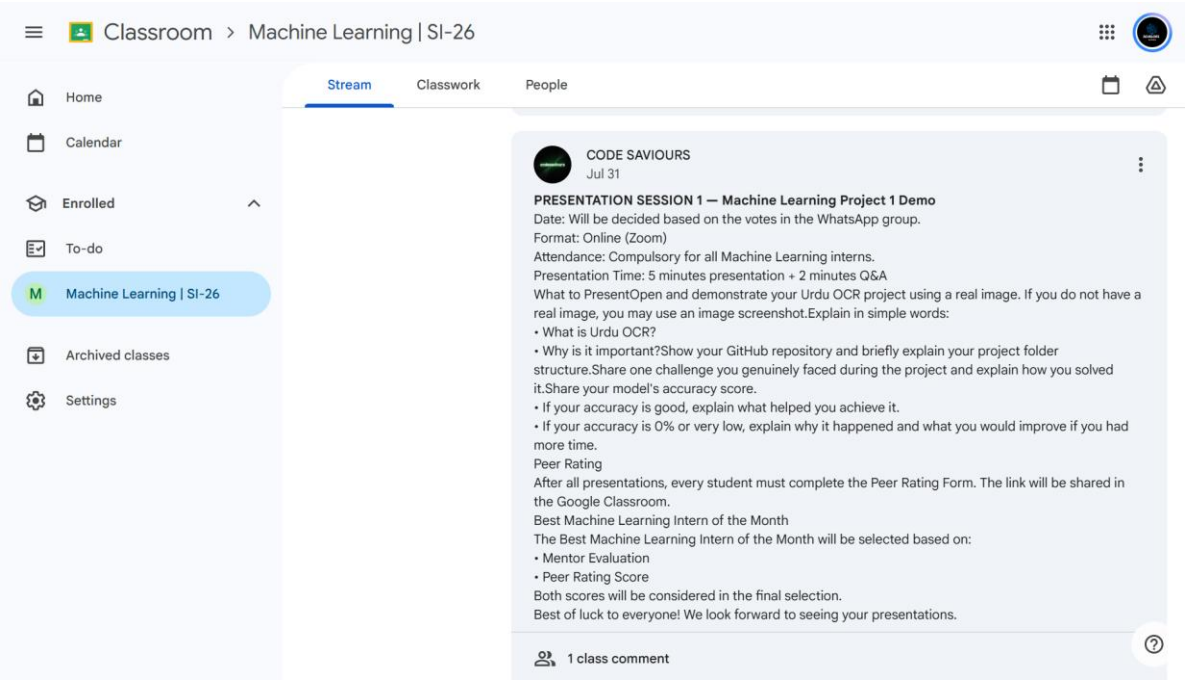

Figure 27: Announcement for Presentation Session 1: Urdu OCR Project Demo

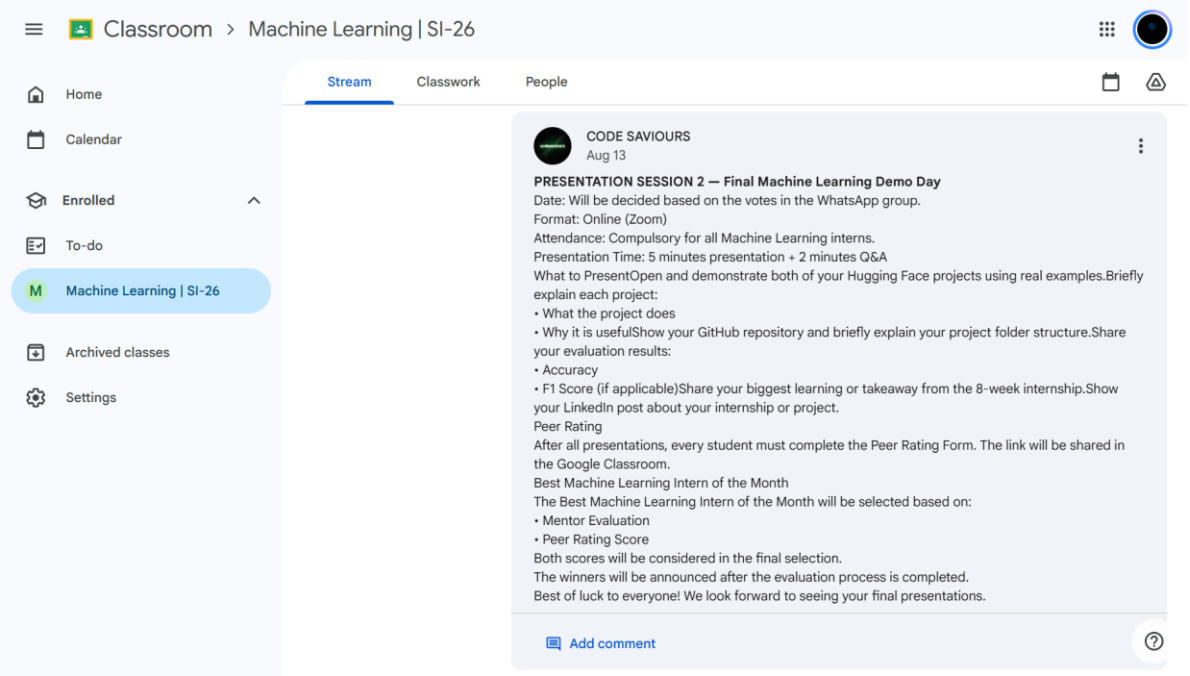

Figure 28: Announcement for Presentation Session 2: Final Machine Learning Demo Day